```python
import time
import multiprocessing as mp
from multiprocessing import shared_memory
import numpy as np
SIZES = [100_000, 1_000_000, 10_000_000]
REPEATS = 3
def ipc_receiver(conn):  1 usage
    try:
        while True:
            data = conn.recv()
            if data is None:
                break
            checksum = int(np.sum(data, dtype=np.int64))
            conn.send(checksum)
    finally:
        conn.close()
def shared_memory_receiver(conn):  1 usage
    shm = None
    try:
        while True:
            message = conn.recv()
            if message is None:
                break
            if message[0] == "attach":
                shm = shared_memory.SharedMemory(name=message[1])
                conn.send("ready")
            elif message[0] == "read":
                array = np.ndarray(
                    message[1],
                    dtype=np.int32,
```

```
                    message[1],
                    dtype=np.int32,
                    buffer=shm.buf
                )
                checksum = int(np.sum(array, dtype=np.int64))
                conn.send(checksum)
    finally:
        if shm is not None:
            shm.close()
        conn.close()

def benchmark_ipc(array, process, conn):  1 usage
    times = []
    for _ in range(REPEATS):
        start = time.perf_counter()
        conn.send(array)
        result = conn.recv()
        elapsed = (time.perf_counter() - start) * 1000
        times.append(elapsed)
        expected = int(np.sum(array, dtype=np.int64))
        assert result == expected, "IPC checksum mismatch"
    return float(np.median(times))
def benchmark_shared(array, shm, process, conn):  1 usage
    times = []

    for _ in range(REPEATS):
        start = time.perf_counter()
        conn.send(("read", array.shape))
        result = conn.recv()
```

```
51      def benchmark_shared(array, shm, process, conn):  1 usage
56              conn.send(("read", array.shape))
57              result = conn.recv()
58              elapsed = (time.perf_counter() - start) * 1000
59              times.append(elapsed)
60              expected = int(np.sum(array, dtype=np.int64))
61              assert result == expected, "Shared memory checksum mismatch"
62          return float(np.median(times))
63
64      def main():  1 usage
65          ctx = mp.get_context("spawn")
66          results = []
67          print("TASK 4 — Host-to-Host IPC Transfer Profile")
68          print("CPU-only mode")
69          print(f"Repeats per size: {REPEATS}")
70          print("\nStarting benchmarks...")
71          for size in SIZES:
72              rng = np.random.default_rng(42 + size)
73              array = rng.integers(
74                  1, 101, size=size, dtype=np.int32
75              )
76              parent_conn, child_conn = ctx.Pipe()
77              process = ctx.Process(
78                  target=ipc_receiver,
79                  args=(child_conn,)
80              )
81              process.start()
82              child_conn.close()
83
84              try:
```

```
64      def main():  1 usage
83
84          try:
85              ipc_ms = benchmark_ipc(array, process, parent_conn)
86              parent_conn.send(None)
87              process.join()
88          finally:
89              parent_conn.close()
90              if process.is_alive():
91                  process.terminate()
92                  process.join()
93
94          shm = shared_memory.SharedMemory(
95              create=True,
96              size=array.nbytes
97          )
98
99          shared_array = np.ndarray(
100             array.shape,
101             dtype=np.int32,
102             buffer=shm.buf
103         )
104         shared_array[:] = array
105
106         parent_conn, child_conn = ctx.Pipe()
107         process = ctx.Process(
108             target=shared_memory_receiver,
109             args=(child_conn,)
110         )
111         process.start()
```

```
64      def main():  1 usage
110         )
111         process.start()
112         child_conn.close()
113
114         try:
115             parent_conn.send(("attach", shm.name))
116             assert parent_conn.recv() == "ready"
117
118             shared_ms = benchmark_shared(
119                 array, shm, process, parent_conn
120             )
121
122             parent_conn.send(None)
123             process.join()
124         finally:
125             parent_conn.close()
126             if process.is_alive():
127                 process.terminate()
128                 process.join()
129             shm.close()
130             shm.unlink()
131
132         results.append((size, ipc_ms, shared_ms))
133
134         print(f"\nN = {size:,}")
135         print(f"Path A — IPC/Pickle: {ipc_ms:.2f} ms")
136         print(f"Path B — Shared Memory: {shared_ms:.2f} ms")
137         print(f"Transfer overhead difference: {ipc_ms - shared_ms:.2f} ms")
138
```

```
64      def main():  1 usage
137         print(f"Transfer overhead difference: {ipc_ms - shared_ms:.2f} ms")
138
139     lines = [
140         "TASK 4 — Host-to-Host IPC Transfer Profile",
141         "Mode: CPU-only",
142         f"Repeats per size: {REPEATS}",
143         "Times are median round-trip times, including receiver checksum.",
144         "",
145         "N | Path A IPC/Pickle (ms) | Path B Shared Memory (ms) | Difference (ms)"
146     ]
147
148     for size, ipc_ms, shared_ms in results:
149         lines.append(
150             f"{size} | {ipc_ms:.2f} | {shared_ms:.2f} | "
151             f"{ipc_ms - shared_ms:.2f}"
152         )
153
154     with open("task4_output.txt", "w", encoding="utf-8") as file:
155         file.write("\n".join(lines) + "\n")
156
157     print("\nResults saved to task4_output.txt")
158
159
160 if __name__ == "__main__":
161     main()
162
```